Code Explanation

Data Loader Module Explanation

This module provides functionality for loading transaction data, customer reference data, and store reference data from various sources using Apache Spark. It supports multiple data formats such as delta, parquet, and csv.

Overview of the Code
The code is a Python module that utilizes Apache Spark for data loading. It contains four primary functions: `get_spark_session`, `load_transactions`, `load_customer_data`, and `load_store_data`. These functions enable the creation of a Spark session and the loading of different types of data from specified sources.

Step 1: Creating a Spark Session
The first step is to create a Spark session using the `get_spark_session` function. This function returns an active Spark session.

```
def get_spark_session() -> SparkSession:
    return (SparkSession.builder
            .appName("TransactionAnalytics")
            .config("spark.sql.extensions", "io.delta.sql.DeltaSpark
SessionExtension")
            .config("spark.sql.catalog.spark_catalog", "org.apache.s
park.sql.delta.catalog.DeltaCatalog")
            .getOrCreate())
```

Step 2: Defining the Schema for Transaction Data
The `load_transactions` function defines a schema for transaction data using `StructType` and `StructField`. This schema includes fields such as `transaction_id`, `customer_id`, `transaction_date`, `amount`, `category`, `store_id`, `payment_method`, and `is_online`.

```
transaction_schema = StructType([
    StructField("transaction_id", StringType(), False),
    StructField("customer_id", StringType(), False),
    StructField("transaction_date", TimestampType(), False),
    StructField("amount", DoubleType(), False),
    StructField("category", StringType(), True),
    StructField("store_id", StringType(), True),
    StructField("payment_method", StringType(), True),
    StructField("is_online", StringType(), True)
])
```

Step 3: Loading Transaction Data Based on Format
The `load_transactions` function loads transaction data from a specified source based on the provided format type. If the format is "csv", it uses the defined schema to read the data. For "delta" or "parquet" formats, it directly loads the data.

```
if format_type == "csv":
    return spark.read.format("csv")
```

```
        .option("header", "true")
        .schema(transaction_schema)
        .load(source_path)
elif format_type in ["delta", "parquet"]:
    return spark.read.format(format_type)
        .load(source_path)
else:
    raise ValueError(f"Unsupported format type: {format_type}")
```

Step 4: Loading Customer Reference Data
The `load_customer_data` function loads customer reference data from a specified source. It defines a schema for customer data and loads the data based on the provided format type.

```
customer_schema = StructType([
    StructField("customer_id", StringType(), False),
    StructField("customer_name", StringType(), True),
    StructField("email", StringType(), True),
    StructField("signup_date", TimestampType(), True),
    StructField("customer_segment", StringType(), True),
    StructField("loyalty_tier", StringType(), True),
    StructField("age_group", StringType(), True)
])

if format_type == "csv":
    return spark.read.format("csv")
        .option("header", "true")
        .schema(customer_schema)
        .load(source_path)
else:
    return spark.read.format(format_type)
        .load(source_path)
```

Step 5: Loading Store Reference Data
The `load_store_data` function loads store reference data from a specified source. It defines a schema for store data and loads the data based on the provided format type.

```
store_schema = StructType([
    StructField("store_id", StringType(), False),
    StructField("store_name", StringType(), True),
    StructField("location", StringType(), True),
    StructField("region", StringType(), True),
    StructField("store_type", StringType(), True)
])

if format_type == "csv":
    return spark.read.format("csv")
        .option("header", "true")
        .schema(store_schema)
        .load(source_path)
else:
    return spark.read.format(format_type)
        .load(source_path)
```